

2Care.ai Assignment Reference Document

Real-Time Multilingual Voice AI Agent

Clinical Appointment Booking System

1. Objective

The goal of this assignment is to build a **Real-Time Voice AI Agent** that can communicate with patients and **manage clinical appointments automatically through voice conversations**.

The AI agent should be able to:

- Understand spoken language from the user
- Convert speech into text
- Interpret the request using an AI model
- Perform appointment-related actions such as:
 - Booking appointments
 - Rescheduling appointments
 - Cancelling appointments
- Speak responses back to the user

The system must support **three languages**:

- English
- Hindi
- Tamil

The AI agent must also maintain **contextual memory**, meaning it remembers:

- The current conversation context
- Past interactions with the patient

Additionally, the system should support **outbound campaigns**, meaning the AI can initiate calls for reminders or follow-ups.

This project demonstrates skills in:

- AI agent architecture
- Real-time voice processing
- Multilingual NLP systems
- System latency optimization
- Tool orchestration
- Backend engineering
- Memory system design
- Real-time communication pipelines

The **target latency requirement** is:

Less than 450 milliseconds from speech end to first audio response.

Latency must be **measured and logged** in the implementation.

2. Technical Requirements

2.1 Programming Languages

Preferred languages:

Language	Purpose
Python	AI services, agent orchestration
TypeScript	API services and application logic

2.2 Core Technologies

Technology	Purpose
WebSockets	Real-time audio communication
FastAPI / Node.js	Backend service
Redis	Memory storage and session data
Whisper / Speech API	Speech-to-text conversion
LLM (OpenAI / Llama / Claude)	Agent reasoning
TTS Engine	Convert text response to speech
PostgreSQL / JSON store	Appointment data
Docker	Deployment

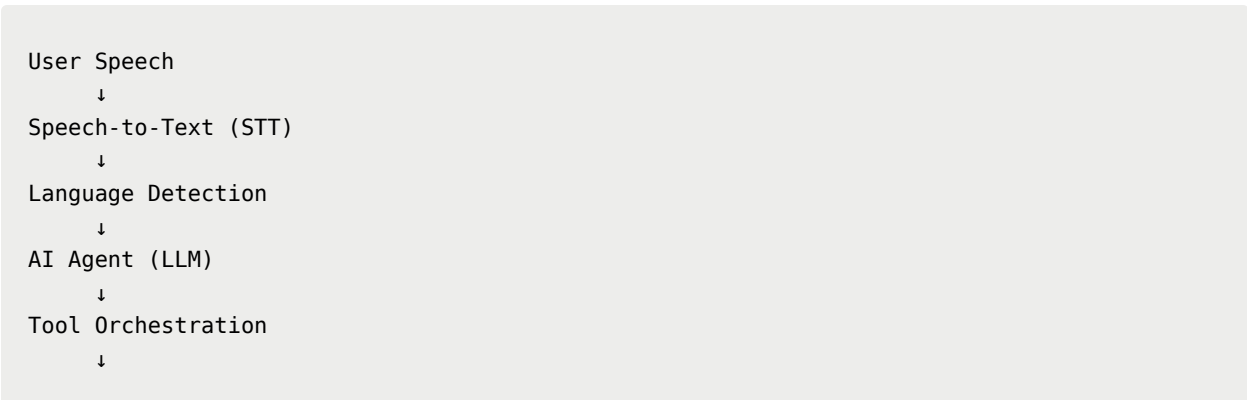
2.3 Required AI Components

The system will use multiple AI modules:

Component	Function
Speech-to-Text	Convert voice input to text
Language Detection	Identify language
LLM Agent	Reason over requests
Tool Orchestration	Call appointment APIs
Text-to-Speech	Convert responses into audio

3. System Architecture Overview

The architecture consists of a **real-time conversational pipeline**.



Appointment Service

↓

Text Response

↓

Text-to-Speech (TTS)

↓

Audio Response

The pipeline must be designed for **low latency**.

4. Project Features

The system must support the **complete appointment lifecycle**.

Core Functionalities

1. Appointment booking
2. Appointment rescheduling
3. Appointment cancellation
4. Conflict detection
5. Doctor availability checking
6. Alternative suggestions

Multilingual Voice Conversation

The agent must support:

Language	Example
English	"Book appointment with cardiologist tomorrow"
Hindi	"मुझे कल डॉक्टर से मिलना है"
Tamil	"நாளை மருத்துவரை பார்க்க வேண்டும்"

The agent should automatically detect the language.

Contextual Memory

The system must store context at two levels.

Session Memory

Stores current conversation context.

Example:

```
User: Book appointment  
Agent: Which doctor?  
User: Cardiologist
```

Session memory tracks the pending intent.

Persistent Memory

Stores long-term user history.

Example:

```
Patient prefers Hindi  
Last appointment: Dr Sharma  
Preferred hospital: Apollo
```

Outbound Campaign Mode

The system should support proactive calling.

Examples:

- Appointment reminders
- Follow-up checkups
- Vaccination reminders

Example conversation:

Agent:

"Hello, this is a reminder about your appointment tomorrow."

Patient:

"Can we move it to Friday?"

Agent should handle this naturally.

5. Step-by-Step Implementation Guide

This section explains **how to build the system from scratch**.

Step 1: Setup Project Environment

Create the project structure.

```
voice-ai-agent
├── backend
├── agent
├── services
├── memory
├── scheduler
├── frontend (optional)
└── docs
```

Step 2: Setup Backend Server

Use **FastAPI (Python)** or **Node.js (TypeScript)**.

Example using FastAPI.

Install dependencies:

```
pip install fastapi uvicorn redis openai
```

Create server file.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
```

```
def health():  
    return {"status": "running"}
```

Run server:

```
uvicorn main:app --reload
```

Step 3: Implement Speech-to-Text

Use Whisper or any speech recognition API.

Example concept:

```
Audio Input  
→ Speech Recognition  
→ Text Output
```

Example output:

```
User speech: "Book appointment tomorrow"  
Converted text: "book appointment tomorrow"
```

Step 4: Language Detection

Use libraries such as:

- langdetect
- fastText
- LLM inference

Example detection:

```
Input: "मुझे डॉक्टर से मिलना है"
```

Detected Language: Hindi

Step 5: Design AI Agent

The agent interprets user intent.

Example prompt design:

```
You are a healthcare appointment assistant.

Available actions:
1 Book appointment
2 Cancel appointment
3 Reschedule appointment
4 Check availability

User request: "I want to see a dermatologist tomorrow"

Return JSON:
{
  "intent": "book",
  "doctor": "dermatologist",
  "date": "tomorrow"
}
```

Step 6: Tool Orchestration

The agent must call backend tools.

Example tools:

Tool	Description
checkAvailability	Check doctor schedule
bookAppointment	Create booking
cancelAppointment	Cancel booking
rescheduleAppointment	Change time

Example tool response:

Available slots:

10:30 AM

2:00 PM

4:30 PM

Step 7: Scheduling Logic

Implement logic to prevent invalid bookings.

Check conditions:

- Doctor available
- Slot not double-booked
- Time not in the past

Example validation:

```
if slot in booked_slots:
    return "Slot already booked"
```

The agent must suggest alternatives.

Step 8: Contextual Memory Design

Use Redis or database.

Example session storage:

```
session_id → conversation_state
```

Example stored context:

```
{
  "intent": "booking",
  "doctor": "cardiologist",
```

```
"date": "tomorrow"
}
```

Persistent memory example:

```
patient_id
preferred_language
past_appointments
```

Step 9: Text-to-Speech

Convert agent response to voice.

Example response:

```
Your appointment with Dr Sharma is booked for tomorrow at 10 AM.
```

Convert to audio output.

Step 10: Real-Time Communication

Use **WebSockets** for low latency.

Workflow:

```
Voice → WebSocket → Backend
Backend → Response → WebSocket → Audio
```

This enables real-time interaction.

Step 11: Latency Measurement

You must measure system latency.

Latency includes:

Stage	Time
Speech recognition	120 ms
Agent reasoning	200 ms
Speech synthesis	100 ms

Total target:

< 450 ms

Log metrics in the system.

6. Project Structure

Recommended structure:

```

voice-ai-agent
├── backend
│   ├── api
│   ├── controllers
│   └── routes
├── agent
│   ├── prompt
│   ├── reasoning
│   └── tools
├── memory
│   ├── session_memory
│   └── persistent_memory
├── services
│   ├── speech_to_text
│   ├── text_to_speech
│   └── language_detection
├── scheduler
│   └── appointment_engine
└── docs

```

7. Database Design

Example appointment schema.

```
appointments
-----
id
patient_id
doctor_id
date
time
status
```

Doctor availability table.

```
doctor_schedule
-----
doctor_id
date
available_slots
```

8. Validation Rules

The system must prevent:

- Double bookings
- Booking past times
- Invalid doctor IDs

Example rule:

```
if appointment_time < current_time:
    reject
```

9. Error Handling

Examples:

LLM Failure

Return safe fallback message.

```
"I'm having trouble processing that request. Could you repeat?"
```

Scheduling Conflict

```
That slot is already booked.  
Available slots are 2 PM and 4 PM.
```

10. Testing Instructions

Test scenarios:

Scenario	Expected Result
Book appointment	Appointment confirmed
Cancel appointment	Booking removed
Reschedule	Slot updated
Language switching	Agent adapts language
Conflict booking	Alternative slot suggested

11. Evaluation Criteria

Area	Weight
Real-time architecture	20%
Agent reasoning	20%
Memory design	15%
Scheduling logic	10%

Area	Weight
Multilingual capability	10%
Performance optimization	10%
Code structure	10%
Documentation	5%

12. Bonus Features

Optional improvements:

- Interrupt handling (barge-in)
 - Redis memory TTL
 - Horizontal scaling
 - Cloud deployment
 - Background campaign scheduler
-

13. Common Mistakes

Avoid:

- Hardcoding responses instead of tool calls
- Storing API keys in code
- Ignoring latency measurement
- Mixing agent logic with scheduling logic

Keep components modular.

14. Submission Guidelines

You must submit the following.

GitHub Repository

Must include:

- Complete project code
 - Clear project structure
 - README with setup instructions
-

Loom Video

Maximum **3 minutes**.

Show:

- System demo
 - Voice interaction
 - Architecture explanation
-

Architecture Diagram

Submit as:

PNG or PDF

The diagram must show:

- STT
 - Agent
 - Memory
 - Tools
 - Scheduling system
-

README Documentation

The README must include:

- Setup instructions
- Architecture explanation

- Memory design
- Latency breakdown
- Trade-offs
- Known limitations